

AWS CERTIFIED SOLUTIONS ARCHITECT – PROFESSIONAL

Projet Capstone

Concevoir, déployer et documenter l'architecture cloud de l'application « My Music »

selon les 6 piliers du AWS Well-Architected Framework

Cours	AWS Architecting – Professional
Format du travail	En binôme (2 étudiants)
Type de livrable	Code, architecture, captures d'écran, documentation, démonstration
Durée indicative	2 semaines (voir jalons, section 7)
Date de publication du sujet	09/08/2026

1. Présentation du projet

My Music est une application web existante de partage et d'écoute de musique, développée en Node.js (serveur HTTP natif, sans framework), avec une base de données MySQL et trois microservices auxiliaires : envoi d'e-mails (SMTP), upload de fichiers audio/covers, et détection du moment de la journée (pour le thème clair/sombre). Elle propose l'inscription et la connexion avec vérification par OTP, le partage de titres, les likes, le suivi des téléchargements et un profil utilisateur.

Aujourd'hui, l'application ne fonctionne qu'en local, orchestrée par Docker Compose : un conteneur MySQL, un conteneur "app" (le serveur principal), et des conteneurs pour les microservices "mail" et "moment". Les fichiers uploadés sont stockés sur un volume Docker local, non partagé entre plusieurs instances de "app" — un frein explicite à la scalabilité horizontale. Un document d'architecture (docs/ARCHITECTURE.md) propose déjà une piste de refonte événementielle (SNS/SQS/Lambda/S3/SES) mais reste au stade de proposition : rien n'est implémenté ni déployé sur AWS.

Votre mission, en tant qu'architecte AWS, est de faire passer cette application d'un prototype local à une solution cloud déployée sur AWS, opérationnelle, sécurisée, résiliente, performante, économique et sobre en ressources — c'est-à-dire alignée sur les 6 piliers du AWS Well-Architected Framework (WAF).

Ce qui vous est fourni

Le code source complet de My Music (serveur, microservices, frontend, schema.sql, Dockerfile, docker-compose.yml, Readme.md et docs/ARCHITECTURE.md). Vous devez partir de cette base : soit en redéploant les composants tels quels sur des services AWS équivalents, soit en les faisant évoluer (conteneurisation, serverless, managé) si vous justifiez ce choix par un ou plusieurs piliers du WAF.

2. Objectifs pédagogiques

À l'issue du projet, chaque binôme doit être capable de :

- analyser une architecture applicative existante et identifier ses limites (couplage fort, absence de haute disponibilité, secrets en clair, stockage non partagé, etc.) ;
- concevoir une architecture cible sur AWS, documentée par un diagramme draw.io, justifiée pilier par pilier du Well-Architected Framework ;
- choisir et mettre en œuvre les services AWS pertinents (réseau, calcul, base de données, stockage, sécurité, observabilité) ;
- déployer réellement l'application sur AWS et démontrer son fonctionnement de bout en bout ;
- documenter les décisions d'architecture (Architecture Decision Records) et les compromis assumés ;
- travailler en équipe de deux avec une répartition claire des responsabilités, tout en restant capables individuellement de défendre l'ensemble du projet.

3. Architecture actuelle (point de départ)

Composants tels qu'ils existent dans le dépôt fourni :

Composant	Rôle	Détails techniques
app (server.js)	Serveur HTTP principal : sert le frontend statique et expose l'API REST	Node.js, http natif, port 3000, healthcheck /healthz
MySQL	Base de données relationnelle	Tables users, musiques, likes, partages, news, nb_telechargements (schema.sql)
mail	Envoi d'e-mails (inscription, OTP...)	SMTP, port 4001, variables SMTP_* en clair dans l'environnement
moment	Détection matin/soir pour le thème UI	Calcul d'heure pur, sans I/O, port 3004
upload	Upload de fichiers audio/covers	busboy, stockage sur volume Docker local, port 4002 (non branché au frontend actuel)
frontend	Interface utilisateur	HTML/CSS/JS statique servi par "app" (index.html, profile.html)

Limites déjà identifiées dans le dépôt (à traiter dans votre architecture cible) :

- un seul environnement, sans séparation dev/prod, sans Infrastructure as Code ;
- stockage des fichiers uploadés sur volume local : ne supporte pas plusieurs instances de "app" en parallèle ;
- couplage HTTP synchrone entre "app" et les microservices "mail"/"moment" : une panne ou une latence de "mail" peut faire échouer une inscription ;
- secrets (mots de passe DB, identifiants SMTP) passés en variables d'environnement en clair (.env) ;
- aucune haute disponibilité, aucune sauvegarde automatisée, aucune observabilité centralisée (logs/métriques/alarmes).

À propos de docs/ARCHITECTURE.md

Ce document propose une piste de découplage événementiel (SNS/SQS/Lambda, upload direct vers S3, SES pour les e-mails). C'est une source d'inspiration légitime, pas une solution imposée : vous êtes libres de la reprendre, de l'adapter ou de proposer une autre architecture cible, à condition de justifier vos choix par les piliers du Well-Architected Framework.

4. Exigences par pilier du Well-Architected Framework

Votre architecture cible et votre déploiement doivent traiter explicitement les 6 piliers suivants. Pour chaque pilier, vous devez identifier au moins une décision d'architecture concrète appliquée à My Music, avec sa justification.

Pilier	Exigences attendues pour My Music
Excellence opérationnelle	Déploiement reproductible (Infrastructure as Code recommandé : CloudFormation, CDK ou Terraform), centralisation des logs applicatifs et des microservices (CloudWatch Logs), tableaux de bord et alarmes sur les métriques clés, procédure de déploiement documentée et idéalement automatisée (pipeline CI/CD).

Sécurité	Principe du moindre privilège (rôles IAM dédiés par service, pas d'utilisateur root/admin partagé), secrets (mots de passe MySQL, identifiants SMTP, clés API) stockés dans AWS Secrets Manager ou Systems Manager Parameter Store — jamais en clair dans le code ou les variables d'environnement commitées, chiffrement au repos et en transit (RDS, S3, HTTPS), segmentation réseau (VPC, sous-réseaux publics/privés, security groups), protection du flux d'authentification et de l'OTP existant.
Fiabilité	Tolérance à la panne d'une zone de disponibilité (Multi-AZ pour la base de données, plusieurs instances de calcul derrière un load balancer), health checks (le endpoint /healthz existant doit être branché à votre mécanisme de supervision), stratégie de sauvegarde et de restauration de la base de données, gestion des pannes des microservices auxiliaires (mail, moment, upload) sans bloquer le parcours utilisateur principal.
Performance	Choix dimensionné des ressources de calcul (type d'instance, taille de tâche Fargate ou équivalent), distribution du contenu statique et des fichiers audio via un CDN (CloudFront) plutôt que de les servir systématiquement depuis le serveur applicatif, stratégie de mise en cache pertinente.
Optimisation des coûts	Dimensionnement au plus juste (right-sizing) des ressources, choix motivé entre options serverless et options provisionnées, classes de stockage S3 adaptées au cycle de vie des fichiers, mise en place d'alertes de budget (AWS Budgets) pour éviter les dérives de coûts durant le projet.
Développement durable	Choix de région et de services cohérent avec une utilisation raisonnée des ressources, suppression des ressources inutilisées après chaque phase de test, préférence pour des services managés/serverless qui mutualisent l'infrastructure plutôt que des ressources dédiées surdimensionnées.

5. Livrables attendus

Chaque binôme doit remettre l'ensemble des livrables suivants. Un livrable manquant ou non fonctionnel entraîne une pénalité sur la grille d'évaluation (section 8).

1. **Code source complet** — dépôt Git contenant le code de l'application (repris et, si nécessaire, adapté à partir de la base fournie), tout code d'Infrastructure as Code utilisé, et un historique de commits reflétant la contribution des deux membres du binôme. Le Readme.md doit être mis à jour pour refléter le déploiement AWS réel (et non plus seulement le fonctionnement local).
2. **Diagrammes d'architecture draw.io** — au minimum deux diagrammes, fournis en fichier source .drawio ET en export image (PNG ou PDF) : (a) l'architecture "AS-IS" représentant le système actuel tel que décrit en section 3, et (b) l'architecture cible "TO-BE" déployée sur AWS, avec une légende explicite des services AWS utilisés, des flux réseau (VPC, sous-réseaux, passerelles) et des flux de données.
3. **Captures d'écran des étapes de mise en œuvre** — captures horodatées de la console AWS (et/ou sorties de l'AWS CLI) démontrant la création réelle des ressources clés à chaque étape majeure : réseau (VPC), sécurité (IAM, Security Groups, Secrets Manager), base de données (RDS), compute (ECS/Fargate, EC2 ou Lambda selon vos choix), stockage (S3), distribution (CloudFront), supervision (CloudWatch). Organisez-les dans le rapport dans l'ordre chronologique du déploiement.
4. **Rapport des étapes de mise en solution** — document décrivant, étape par étape, comment vous êtes passés de l'architecture actuelle à l'architecture cible : décisions prises, alternatives envisagées et écartées (avec la raison), difficultés rencontrées et solutions apportées, et justification explicite par pilier du Well-Architected Framework (voir section 4).
5. **Solution finale démontrable** — l'application My Music doit être accessible et fonctionnelle sur AWS au moment de la soutenance (URL publique ou accès fourni à l'évaluateur), avec les parcours clés opérationnels : inscription/connexion avec OTP, partage d'un titre, like, téléchargement, consultation du profil. Si les ressources doivent être détruites avant la soutenance pour des raisons de coût, une vidéo de démonstration complète et récente est acceptée en remplacement, avec justification.
6. **Soutenance orale** — présentation de 15 à 20 minutes devant la classe (architecture, démonstration, retour d'expérience), suivie de questions. Les deux membres du binôme doivent présenter et être en mesure de répondre à des questions sur n'importe quelle partie du projet, y compris celle principalement portée par l'autre membre.

6. Organisation du travail en binôme

Le projet est réalisé par équipes de deux étudiants. Une répartition des responsabilités est recommandée pour organiser le travail, mais elle n'exonère aucun des deux membres de comprendre et de savoir défendre l'ensemble de la solution.

Répartition suggérée (à adapter librement) :

- Membre A — référent "Infrastructure & Sécurité" : réseau (VPC), IAM, secrets, base de données, observabilité, coûts.
- Membre B — référent "Application & Continuité de service" : conteneurisation/déploiement applicatif, microservices, stockage des fichiers, CDN, tests de résilience.

Le rapport (livrable n°4) doit mentionner explicitement qui a porté quelle partie, et inclure une section courte de rétrospective binôme (ce qui a bien fonctionné dans la collaboration, ce qui a été difficile).

7. Démarche suggérée et jalons

Cette progression est indicative : adaptez-la à votre calendrier réel, mais gardez la logique "comprendre → concevoir → construire → vérifier → présenter".

Jalon	Objectif	Livrable(s) associé(s)
J1 — Analyse	Prendre en main le code fourni, exécuter l'application en local (docker-compose), lister les manquements par pilier WAF sur l'architecture actuelle.	Diagramme AS-IS (draw.io), liste de constats
J2 — Conception	Concevoir l'architecture cible AWS, choisir les services, produire le diagramme TO-BE, faire valider les choix (justification par pilier).	Diagramme TO-BE (draw.io), premières sections du rapport
J3 — Fondations	Mettre en place le réseau (VPC, sous-réseaux, security groups), IAM, Secrets Manager, et la base de données (RDS) avec import de schema.sql.	Captures d'écran réseau/sécurité/RDS
J4 — Déploiement applicatif	Déployer "app" et les microservices (mail, moment, upload ou leur équivalent AWS), configurer le stockage des fichiers (S3) et la distribution (CloudFront).	Captures d'écran compute/stockage/CDN, application accessible
J5 — Observabilité & fiabilité	Mettre en place logs et alarmes CloudWatch, tester la panne d'un composant, vérifier sauvegardes RDS, revoir les coûts (AWS Budgets).	Captures d'écran monitoring, tests de résilience documentés
J6 — Finalisation	Revue finale (AWS Well-Architected Tool si accessible), finaliser le rapport, préparer et répéter la maintenance, nettoyer les ressources non nécessaires.	Rapport complet, solution démontrable, support de maintenance

8. Grille d'évaluation

Barème sur 100 points.

Critère	Pilier(s) WAF concerné(s)	Points
Excellence opérationnelle (IaC, logs, supervision, procédure de déploiement)	Excellence opérationnelle	15
Sécurité (IAM, secrets, chiffrement, segmentation réseau)	Sécurité	20
Fiabilité (haute disponibilité, sauvegardes, tolérance de panne)	Fiabilité	15

Performance (dimensionnement, CDN, mise en cache)	Performance	15
Optimisation des coûts (right-sizing, alertes budget)	Optimisation des coûts	10
Développement durable (sobriété des ressources, nettoyage)	Développement durable	5
Qualité des livrables (diagrammes draw.io, captures d'écran, rapport)	Transverse	10
Soutenance (clarté, démonstration réelle, répartition binôme)	Transverse	10
Total		100

9. Contraintes et recommandations techniques

- Utiliser un compte AWS dédié au projet (Free Tier ou crédits fournis par l'établissement) ; ne jamais utiliser de compte de production réel.
- L'usage d'un outil d'Infrastructure as Code (CloudFormation, AWS CDK ou Terraform) est fortement recommandé et valorisé dans l'excellence opérationnelle ; un déploiement manuel via la console est accepté mais doit être intégralement documenté par captures d'écran.
- Aucun secret (mot de passe, clé d'accès, jeton API) ne doit apparaître en clair dans le code source déposé ou dans les captures d'écran fournies.
- Les ressources AWS doivent être supprimées (ou arrêtées) après la soutenance pour éviter toute facturation résiduelle ; conserver les captures d'écran comme preuve du travail réalisé.
- Le binôme reste libre de faire évoluer l'implémentation technique (par exemple conteneuriser différemment, remplacer un microservice par une fonction Lambda) tant que le comportement fonctionnel décrit dans le Readme.md original est conservé et que chaque écart est justifié.

10. Modalités de remise

- Dépôt d'une archive ou d'un lien de dépôt Git contenant : le code, les fichiers .drawio et leurs exports image, le rapport (docx ou pdf) et les captures d'écran.
- Nom du dossier/de l'archive : capstone_mymusic_<nom1>_<nom2>.zip
- La soutenance est planifiée par l'enseignant ; toute absence non justifiée d'un membre du binôme entraîne une pénalité individuelle.

Annexe A — Rappel des 6 piliers du Well-Architected Framework

Excellence opérationnelle — Exploiter et surveiller les systèmes pour délivrer de la valeur, et améliorer continuellement les processus.

Sécurité — Protéger les informations, les systèmes et les actifs tout en délivrant de la valeur métier via une évaluation des risques et des stratégies de mitigation.

Fiabilité — Assurer qu'un système accomplit sa fonction correctement et de façon cohérente, y compris la capacité à récupérer d'une panne.

Performance — Utiliser les ressources informatiques de façon efficace pour répondre aux exigences du système, et maintenir cette efficacité face à l'évolution des besoins.

Optimisation des coûts — Éviter les coûts inutiles en comprenant et en maîtrisant les dépenses.

Développement durable — Minimiser les impacts environnementaux des charges de travail cloud exécutées.

Annexe B — Documents fournis dans le dépôt

- Readme.md — description fonctionnelle et démarrage local de l'application
- docker-compose.yml et Dockerfile — orchestration locale actuelle des 4 conteneurs (mysql, app, mail, moment) et du microservice upload
- schema.sql — structure de la base de données MySQL (users, musiques, likes, partages, news, nb_telechargements)
- docs/ARCHITECTURE.md — proposition (non implémentée) de refonte événementielle SNS/SQS/Lambda/S3/SES
- docs/my_music-HLD.png — schéma haut niveau existant, à titre de référence